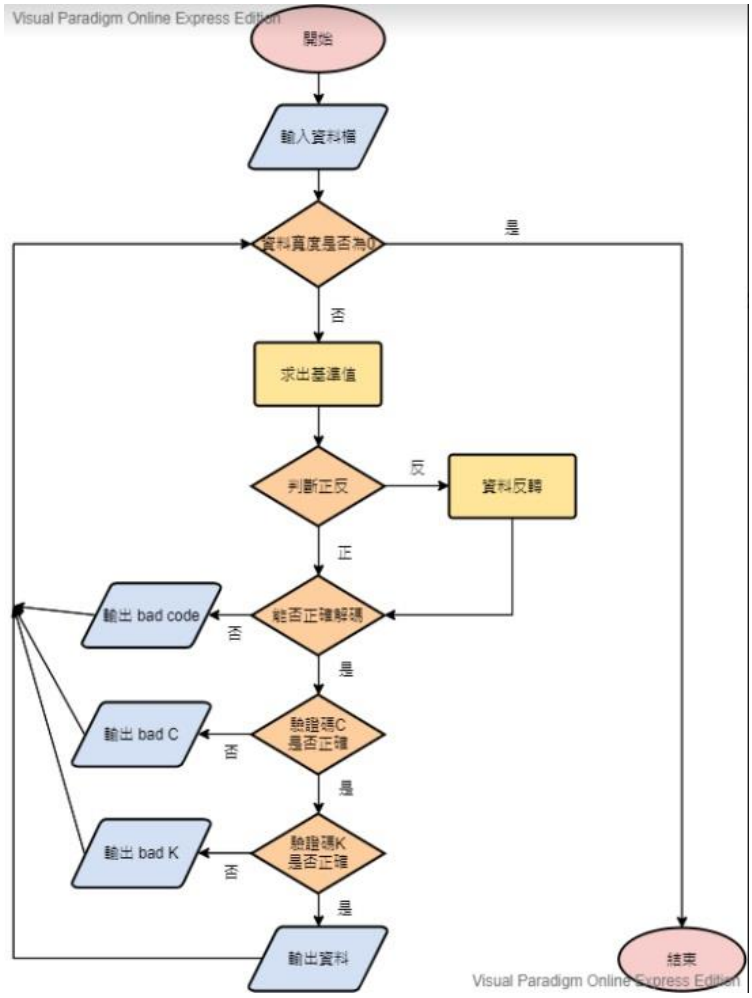
前言

這是由 B10915065 李昶勳以及 B10915061 詹宗叡所完成的條碼解碼器，依照 Code-11 標準，Code-11 是一種條碼系統，主要用於標籤電信設備，編碼的字元僅限於數字 0 到 9，破折號（-）和特殊字元出現在每個 code-11 條碼開頭和結尾的開始，Code-11 是一個離散系統，每個字元都獨立編碼。字元的編碼由五個在深色和淺色之間交替的相鄰區域或條形表示，從黑條開始，每個條形的寬度是窄或是寬。

流程圖



解碼方式

我們利用二進位的概念來辨別每個 Character，每次抓一個寬度進來判斷窄寬。

窄 0 寬 1，寬就加上權重，變數 count 計數 5 次就判斷一次。

Character	Encoding	十進位
0	00011	3
1	01001	9
2	10000	16
3	10100	20
4	11000	24
5	00101	5
6	01100	12
7	00001	1
8	10010	18
9	10001	17
-	00100	4
Start/Stop	00110	6

如果不是抓出來五個算出來的和不是表上其中一個十進位數字，就代表是 bad code，抓完五個後要判斷下一個寬度是不是窄的，如果是寬的也代表是 bad code，直到遇到檢查碼 C 的前一個位元。

而在程式一開始使用者輸入測資後，我們先判斷前五個以及後五個條碼是不是正常的開始以及結束字元，因為要反的 code 也能處理，我們順便判斷正反，如果方向是反的，我們就把讀入的資料反過來，再丟入解碼函式

誤差值範圍處理

而因為印刷問題，大小還有可能會有 5%的誤差，這部份我們是利用了在一開始讀入測資之後，我們又開一個新的陣列去把讀進來的寬度由小排到大，如果要能解碼一定要滿足 0.95*standard<=最小寬度，standard 指的是預設寬度，我們能利用前面的不等式去計算 standard 可能最大為多少，值越大範圍比較大，所以我們只看最小寬度/0.95，寬度只會是整數所以算出來取 int，去看能不能把使用者輸入的值能不能滿足以下條件

1.standard*2*0.95<=寬最小

2.寬最大(最後一個元素)<=standard*2*1.05

如果上面有一個沒有辦法達成，就代表是 bad code。

而在計算下限的時候，算出來有小數點就無條件進位，因為如果算出來是 32.3 實際上最小就只能是 33。

而上限只需要把小數點捨去就好

抓使用者輸入寬最小的方式是使用最小的寬度計算出寬度為窄的最大可能性 w，用迴圈去找到 v[index]<=可能出現最大寬度 w 且 v[index+1]>w，因為 code 一定會出現寬，如果找不到就是 bad code。

以上都完成之後我們之後只要把寬度為窄的最大可能性回傳，以後判斷窄寬都利用那個值 v，如果寬度<=v 就代表是窄的，反之為寬。

分工:

B10915065 李昶勳:完成程式、書面報告

B10915061 詹宗叡:完成測資產生器協助 debug、書面報告

心得:

B10915065 四資工一 李昶勳

這個專題，令我們困擾許久的問題就是誤差值範圍，我和我的組員討論了許久，最後才討論出了一個比較好的解決方式。

而我認為我收穫最大的就是學會如何與人合作完成一個專案，在這之前基本上都是自己一個人去完成一個程式，但與人合作就必須考慮到對方的想法，把兩個人的想法結合成一個，結合的過程中可能會遇到對方不太瞭解自己的想法，就得細心的再向他解釋一遍。

B10915061 詹宗叡

經過這次的專案，我學到如何與人共同完成一個產品，因為每一個人的想法都是不一樣的，所以我們需要不斷的討論以及測試來找出最好的演算法，溝通是一件非常重要的事，要有溝通才能一起完成目標。還有這次寫完專案也讓我了解了 Code-11 這個編碼方式，以往我只知道超商的條碼，但是卻完全不知道它的原理，這次的專案讓我學了不少的東西。

原始碼:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
int direction(int *,int,int);
void decode(int *,int,int,int);
int checkC(char *,char,int);
int checkK(char *,char,int,char);
void reverse(int *,int);
```

```
void quicksort(int *,int,int);
int partition(int *,int ,int);
void swap(int *,int *);
int    deal(int*,int);
int main(int argc,char *argv[]){
        int m,time=0;
        int *d,*second,maxWidthofnarrow;
        FILE* ptr;
        if((ptr = fopen(argv[1],"r")) == NULL){
                puts("File could not be opened");
        }else{
                fscanf(ptr,"%d",&m);
                while(m!=0){
                        time++;
                        if(m>150||(m-29)%6 !=0){ // bad code
                                int * temp = (int*)malloc(sizeof(int)*(m+1));
                                for(int j =1;j<=m;j++){
                                        fscanf(ptr,"%d",&temp[j]);
                                }
                                printf("Case %d: bad code\n",time);
                                free(temp);
                                fscanf(ptr,"%d",&m);
                                continue;
                        }
                        d = (int*)malloc(sizeof(int)*(m+1) );
                        second= (int *)malloc(sizeof(int)*(m+1));
```

```
for(int i =1;i<=m;i++){
        fscanf(ptr,"%d" ,&d[i]);
        second[i] = d[i];
}
quicksort(second,1,m);//ascending
int status2 = deal(second,m);
free(second);
if(status2 !=-1){
        maxWidthofnarrow = status2;
}else{
        printf("Case %d: bad code\n",time);
        free(d);
        fscanf(ptr,"%d",&m);
        continue;
}
//
int status = direction(d,maxWidthofnarrow,m);
/*direction */
switch(status){
        case 1:
                decode(d,maxWidthofnarrow,m-17,time);
                break;
        case 0:
                reverse(d,m);
                decode(d,maxWidthofnarrow,m-17,time);
                break;
```

```
                        default:
                                printf("Case %d: bad code\n",time);
                }
                fscanf(ptr,"%d",&m);
                free(d);
            }
            fclose(ptr);
        }
        return 0;
}
int direction(int *d,int width,int m){
        if((d[1]<=width )&& (d[2]<=width) &&d[3]>width&&d[4]>width &&
d[5]<=width){
            if(d[m-4]<=width&&d[m-3]<=width &&d[m-2]>width&&d[m-1]>width&&
d[m]<=width){
                return 1;
            }
        }
        if((d[1]<=width) && d[2]>width&&d[3]>width&&d[4]<=width&&d[5]<=width){
            if(d[m-4]<=width &&
d[m-3]>width&&d[m-2]>width&&d[m-1]<=width&&d[m]<=width){
                return 0;
            }
        }
        return -1; //error
}
```

```
void decode(int *d,int width,int m,int time){
	char s[100]="";
	int count=0,sum=0,weight=16,index=0,count2=0;
	if(d[6]<=width){
		for(int i =7;i<=m;i++){
			count2++;
			if(count2%6==0){
				if( !(d[i]<=width)){
					printf("Case %d: bad code\n" , time);
					return ;
				}
				continue;
			}
			count++;
			if(d[i]<=width){
				//do nothing
			}else if(d[i]>width){
				sum+=weight;
			}
			weight/=2;
			if(count%5 == 0){
				switch(sum){
					case 3:
					s[index] = '0';
					break;
					case 9:
```

```
s[index] ='1';
break;
case 16:
s[index] ='2';
break;
case 20:
s[index] = '3';
break;
case 24:
s[index] = '4';
break;
case 5:
s[index] = '5';
break;
case 12:
s[index] = '6';
break;
case 1:
s[index] = '7';
break;
case 18:
s[index] = '8';
break;
case 17:
s[index] = '9';
break;
```

```
                case 4:
                s[index] = '-';
                break;
                default:
                printf("Case %d: bad code\n",time);
                return;
            }
            index++;
            weight=16;
            sum=0;
        }
    }
    count=0,sum=0,weight=16,count2=0;
    //check character
    char c,e;
    for(int i=m+1;i<=m+12;i++){
        count2++;
        if(count2 %6==0){
            if(!(d[i]<=width) ){
                printf("Case %d: bad code\n",time);
                return;
            }
            continue;
        }
        count++;
        if(d[i]<=width){
```

```
        //do nothing
}else if(d[i]>width){
        sum+=weight;
}
weight/=2;
if(count%5==0){
          switch(sum){
                case 3:
                c = '0';
                break;
                case 9:
                c ='1';
                break;
                case 16:
                c ='2';
                break;
                case 20:
                c = '3';
                break;
                case 24:
                c = '4';
                break;
                case 5:
                c = '5';
                break;
                case 12:
```

```
        c = '6';
        break;
        case 1:
        c = '7';
        break;
        case 18:
        c = '8';
        break;
        case 17:
        c = '9';
        break;
        case 4:
        c = '-';
        break;
        default:
        printf("Case %d: bad code\n",time);
        return ;
}
if(count==5){
        if(checkC(s,c,index) !=1 ){
                printf("Case %d: bad C\n",time);
                return ;
        }
        e=c; // record c check character
}else{
        if(checkK(s,c,index,e) !=1){
```

```
                        printf("Case %d: bad K\n",time);
                        return;
                    }
                }
                weight=16;
                sum=0;
            }
        }
        s[index] = '\0';
        printf("Case %d: %s\n" , time,s);
    }else{
        printf("Case %d: bad code\n",time);
        return;
    }
}
int checkC(char * s,char c,int n){
    int correct,sum=0,weight;
    if(c=='-'){
        correct = 10;
    }else{
        correct = c-'0';
    }
    for(int i=1;i<=n;i++){
        if(s[i-1] == '-'){
            weight = 10;
        }else{
```

```
                weight = s[i-1] - '0';
            }
            sum+=((n-i)%10+1)*weight;
        }
        if(sum%11==correct){
            return 1;
        }else{
            return 0;
        }
}
int checkK(char *s,char c,int n,char Ccheck){ // c represent the user enter ,Ccheck is
last time we calculated
        int sum=0,weight,correct;
        if(c=='-'){
            correct = 10;
        }else{
            correct = c-'0';
        }
        for(int i =1;i<=n+1;i++){
            if(i==n+1){
                if(Ccheck =='-'){
                    weight = 10;
                }else{
                    weight = Ccheck-'0';
                }
            }else{
```

```
                if(s[i-1]=='-'){
                        weight = 10;
                }else{
                        weight = s[i-1] - '0';
                }
        }
        sum+= ((n-i+1)%9+1)*weight;
    }
    if(sum%11 == correct){
        return 1;
    }else{
        return 0;
    }
}
void reverse(int *f,int m){
    for(int j =1;j<m;j++,m--){
        swap(&f[j],&f[m]);
    }
}
int partition(int *arr,int left,int right){
    int pivot= arr[right];
    int index = left-1;
    for(int j=left;j<right;j++){
        if(arr[j]<pivot){
            index++;
            swap(&arr[j],&arr[index]);
```

```
            }
        }
        index++;
        swap(&arr[index],&arr[right]);
        return index;
}
void quicksort(int * arr,int left ,int right){
        if(left<right){
            int index = partition(arr,left,right);
            quicksort(arr,left,index-1);
            quicksort(arr,index+1,right);
        }
}
void swap(int *s,int *f){
        int temp = *s;
        *s = *f;
        *f = temp;
}
int deal (int * v,int size){
        int index=-1,bound = v[1];
        int maxofnarrow = (bound*105)/95;
        for(int i = 1;i<size;i++){ // find narrow max
            if(v[i]<=maxofnarrow&& v[i+1]> maxofnarrow ){
                index    = i;
            }
        }
```

```
    if(index==-1){
        return -1;
    }
    int standard = (bound*100)/95;
    int wide = standard*2;
    double w =    (double) wide*0.95;
    w = ceil(w);
    if(w <=v[index+1] && (int)wide*1.05>=v[size] ){ // wide*0.95
        return maxofnarrow;
    }
    return -1;//bad code
}
```